

Building Autonomous AI Agents

OpenClaw, Hermes & NanoClaw

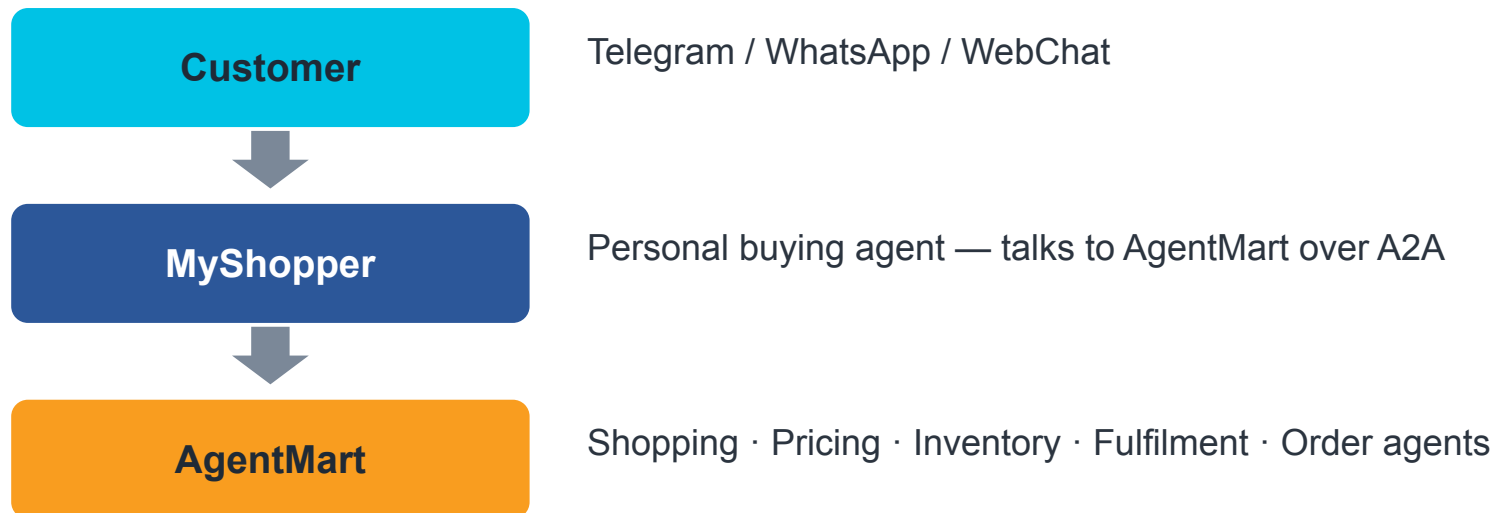
Orchestration · Models · Edge · Heartbeats · Skills · Channels · A2A

Audience: working professionals / graduate learners

Where Today Fits: The Three-Day Progression



Day 3 (today): build **MyShopper**, a personal buying agent owned by the customer — it represents the **customer**, not AgentMart.



Full-Day Agenda

Time	Segment
0:00 – 0:15	Opening & framing
0:15 – 1:00	Part 1: OpenClaw — the orchestration layer
1:00 – 1:45	Part 2: Hermes — the model layer
1:45 – 1:55	Break
1:55 – 2:40	Part 3: NanoClaw — the edge layer
2:40 – 3:40	Part 4: Heartbeat mechanisms (deep dive)
3:40 – 3:50	Break
3:50 – 4:50	Part 5: Skill writing (deep dive)
4:50 – 5:00	Break
5:00 – 6:00	Part 6: Channel integration — messaging & voice
6:00 – 7:00	Part 7: A2A setup — a one-of-its-kind protocol
7:00 – 7:20	Combined lab: end-to-end channel-to-skill flow over custom A2A
7:20 – 7:30	Wrap-up & Q&A

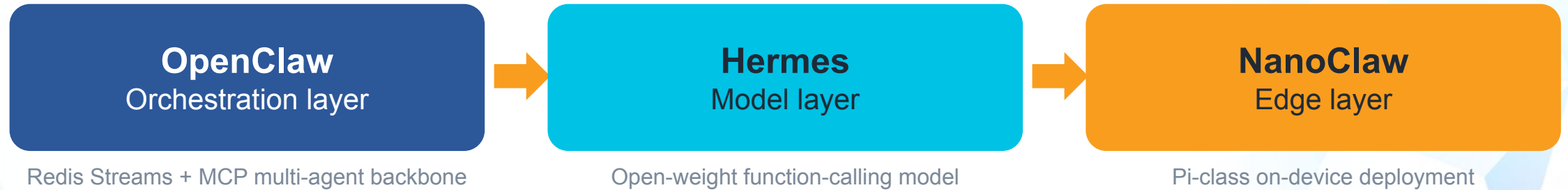
Can also split into three sessions: A = Parts 1–3 · B = Parts 4–5 + lab · C = Parts 6–7 + final lab.

Learning Objectives

By the end of today, you will be able to:

- **Explain** how OpenClaw (orchestration), Hermes (model layer) and NanoClaw (edge deployment) fit together
- **Design a heartbeat / liveness mechanism** for agents — including edge agents on unreliable networks
- **Write agent skills** whose descriptions reliably trigger correct invocation, with basic security hardening
- **Wire an agent to real-world channels** — messaging (Telegram / WhatsApp) and voice — and handle their distinct failure modes
- **Design a custom agent-to-agent (A2A) protocol** on Redis Streams + MCP, distinct from off-the-shelf A2A implementations
- **Build an end-to-end flow:** user messages/speaks → orchestrator dispatches to a healthy edge agent over custom A2A → skill invocation → reply over the same channel

The Stack at a Glance



Four cross-cutting concerns make the stack trustworthy in production:



By the end of the day you will build a small end-to-end system touching every one of these pieces.

The Three Layers: a Comparison

	OpenClaw	Hermes	NanoClaw
Role in the stack	Orchestration layer	Model layer	Edge layer
What it is	Redis Streams + MCP multi-agent backbone	Open-weight function-calling model	Slimmed OpenClaw + Hermes runtime on Pi-class hardware
Why it matters	Durable, replayable, multi-consumer messaging between agents	Cost, latency, no vendor lock-in; structured tool calls as first-class output	Privacy, offline operation, no per-call cost; data stays on device
Watch out for	Stream-topology design + Redis operational overhead	Smaller open models still trail frontier-model quality	Tight RAM, quantization, slower on-device inference

One stack, three layers — the rest of today is about making them trustworthy: heartbeats, skills, channels, A2A.

Reference: Raphael De Lio, “OpenClaw is Going Enterprise — RFC-27: Redis Agent File System”, Medium medium.com/redis-with-raphael-de-lio/openclaw-is-going-enterprise-rfc-27-redis-agent-file-system-dfe256c3b12f

PART 1 · 45 MIN

OpenClaw — the Orchestration Layer

Multi-agent architecture on Redis Streams + MCP, and where the ecosystem plug-ins attach.

OpenClaw: Multi-Agent Architecture

- **Redis Streams + MCP** are the backbone for agent-to-agent communication
 - Redis Streams give **durable, replayable, multi-consumer** messaging between agents
 - MCP standardises how agents expose and call tools
- **Live demo:** spin up two agents talking to each other via Redis Streams
 - Watch: a message published by Agent A appears in Agent B's inbox stream and is consumed independently
- **Ecosystem plug-ins at this layer:**
 - **ClawHub** — the skill marketplace (where reusable skills are published & discovered)
 - **MoltGuard** — the guardrail proxy (policy & safety checks in the request path)



PART 2 · 45 MIN

Hermes — the Model Layer

Why an open-weight function-calling model is the right brain for self-hosted and edge agents.



Hermes: Open-Weight Function Calling

Why open-weight matters

- **Cost** — no per-token API bill at agent scale
- **Latency** — runs next to (or on) the agent; no network round-trip to a hosted API
- **No vendor lock-in** — self-hosted, portable to edge hardware
- **Function calling built in** — structured tool use is a first-class output, not a prompt hack

Hermes vs Claude / GPT tool calling

- Compare Hermes' **tool-calling output format** against Claude / GPT function calling
- Key check: does it emit **OpenAI-style** ``tool_calls``? This decides whether agent frameworks need a translation shim
- **Hands-on:** get Hermes making a structured tool call through an OpenClaw agent

Hermes Core Features: the Automation Engine

- **Natural-language cron** — “every weekday at 9am, summarize my inbox and post to Slack” is all it takes; agents can also **self-schedule wake-ups**; **script mode** (`--script --no-agent`) runs with **zero LLM tokens**
- **Kanban coordination** — a shared Kanban board lets 7+ agents manage themselves: the board reads each profile’s **SOUL.md** for name + role; tasks flow proposed → in-progress → done without you relaying anything
- **Subagent delegation** — talk to **one lead agent**; it delegates to specialist subagents (e.g. a 9-role dev team). Authority is enforced by **toolset restrictions**, not just system prompts; the orchestrator grades work **pass / rework / failed**
- **Self-improving skills** — writes its own skill files after solving hard problems; reusing a skill updates it from what it learns (**deep dive in Part 5**)
- **Memory & profiles** — three-layer memory; separate profiles with their own credentials, memory and scope
- **Safety & control** — approval gates, **/rollback** (rewind files + context), **/steer** (inject instructions mid-run) → **Part 5 security**

Hermes in the Wild: Automation Stories

Cron in production

- **3am “Dreaming” job** — 28 cron jobs, 30+ skills; every night it reads the day’s conversations → decisions, bugs, mistakes → writes tomorrow’s context
- **Gumroad** — cron jobs check support tickets, watch X mentions, update finance docs; every mistake becomes a **dated policy rule** the agent must read
- **20 cron “signals”** — skills log to a daily ledger; an agentic triage job highlights what matters

Agent teams in production

- **7 agents, one Kanban** — given a shared board, they started managing themselves
- **Hermes Swarm** — teams of agents 24/7, messaging peer-to-peer, delegating work, self-scheduling wake-ups
- **12 instances in parallel** — Nous Research runs 12 Hermes instances daily to build Hermes itself

Reference: Hermes Agent Docs — User Stories & Use Cases (326 stories, 15 categories): hermes-agent.nousresearch.com/docs/user-stories

Hermes Deep Cuts: LSP, Curator, SOUL.md & Media

- **LSP — semantic diagnostics** — Hermes runs ~20 real language servers (pyright, typescript-language-server, rust-analyzer, gopls, clangd...) as background subprocesses; after every ``write_file`` / ``patch`` the agent sees only the **new** diagnostics its edit introduced — type errors, undefined names, missing imports. The same architecture top-tier coding agents use, shipped self-contained
- **Curator** — background maintenance for agent-created skills: usage telemetry, **active** → **stale** → **archived** lifecycle, optional LLM consolidation of near-duplicates, snapshots + rollback, **pin** to protect — keeps the self-improvement loop from polluting the skill catalog
- **Personality & SOUL.md** — ``SOUL.md`` is **slot #1 of the system prompt** — the agent's identity; loaded only from ``~/hermes`` so it stays predictable; ``/personality`` presets (teacher, technical, creative, pirate...) as session overlays. Rule of thumb: **SOUL.md = who the agent is; AGENTS.md = how the project works**
- **Image & video generation** — **image generation built in** via FAL.ai (11 models: FLUX 2, GPT-Image, Ideogram V3, Recraft V4...); **video generation** via skills & integrations (e.g. Higgsfield UGC ad studio) — the same tool-extensibility story as MCP

Hermes Persistent Memory

- **Two curated files** — `MEMORY.md` (agent's notes: environment, conventions, lessons — 2,200 chars) + `USER.md` (your profile: preferences, style — 1,375 chars), stored in `~/.hermes/memories/`
- **Frozen snapshot** — injected into the system prompt once at session start (preserves the prefix cache); the `memory` tool (**add / replace / remove**) writes through to disk immediately — visible next session
- **Bounded by design** — hard char limits; when full, the agent **consolidates entries in the same turn**; duplicates rejected; every entry scanned for injection / exfiltration (it lands in the system prompt)
- **`session\_search`** — FTS5 full-text search over **all past sessions** (SQLite), free — no LLM call. Memory = key facts always in context; session search = on-demand “did we discuss X last week?”
- **Consent controls** — `write\_approval` stages saves for review (`/memory pending`); the background self-improvement review saves lessons post-turn; `/journey` shows the learning timeline

Honcho Memory: Dialectic User Modeling

- **Beyond key-value memory** — Honcho is an AI-native memory backend (a memory-provider plugin) that **reasons about conversations after they happen**, building a running model of who the user is — preferences, goals, patterns
- **Dialectic reasoning** — after each turn (cadence-gated), Honcho derives insights; optional **multi-pass depth (1–3): assess** → **self-audit** → **reconcile**; conclusions accumulate server-side
- **Two-layer context injection** — base context (session summary + user representation + peer card) refreshed per turn, plus the dialectic supplement — “what matters right now”
- **Multi-agent isolation** — per-peer profiles: your coding assistant and your personal assistant each see only their own observations — **no cross-contamination**
- **One of 8 memory providers** — Honcho, Mem0, Hindsight, Supermemory, OpenViking, RetainDB, ByteRover, Holographic — all run **alongside** built-in memory, never replace it; setup: ``hermes memory setup``

Hermes Plugins: Extend Everything

- **Drop-in extensibility** — a directory in `~/hermes/plugins/` with `plugin.yaml` + a `register(ctx)` — your tools appear alongside built-ins, no core-code changes
- **Wide surface** — tools, **27 lifecycle hooks** (pre/post tool call, kanban events...), slash + CLI commands, bundled skills, gateway platforms, **image-gen & video-gen backends**, memory providers, context engines, model providers, approval transports, MCP calls
- **Four plugin types** — general (multi-select) · memory providers (one active) · context engines (one active) · model providers (pick one)
- **Opt-in by default** — third-party plugins never run until allow-listed (`plugins.enabled`); bundled infrastructure loads automatically
- **Supply-chain security** — install-time static scanning (safe / caution / **dangerous = blocked**), SHA-pinned installs + curated catalog, **capability consent** with re-consent on update — the OWASP posture from Part 5

PART 3 · 45 MIN

NanoClaw — the Edge Layer

Running agents on Raspberry Pi-class hardware — and what you give up versus the cloud.



NanoClaw: Agents at the Edge

Pi-class hardware constraints

- **Memory** — tight RAM budgets force small models
- **Quantization** — Hermes runs as a quantized build to fit
- **Latency** — on-device inference trades speed for independence
- **Demo:** deploy a slimmed-down OpenClaw + Hermes agent on NanoClaw

Edge vs cloud-hosted (Bedrock / Claude)

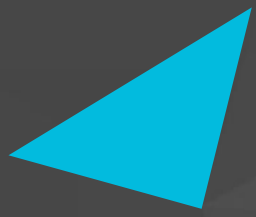
- **Edge wins:** privacy, offline operation, no per-call cost, data stays on device
- **Cloud wins:** model quality, throughput, zero hardware management
- This trade-off returns in **Part 6** — voice latency budgets force the cloud-vs-local choice
- And in **Part 4** — edge connectivity breaks naive heartbeat assumptions



PART 4 · 60 MIN

Heartbeat Mechanisms — Deep Dive

Liveness, readiness and health for agents — including edge agents on unreliable networks.



4.1 Three Signals That Are Often Conflated

- **Liveness** — is the process running at all?
- **Readiness** — can it currently accept work?
- **Health** — is it running within acceptable performance / error bounds?

A process can be alive but not ready, or ready but unhealthy — your detection design must say which signal it actually tracks.

4.1 Push vs Pull Heartbeats

PUSH — agent pings orchestrator

- Agent proactively sends heartbeats on an interval
- + Lower orchestrator load
- – Agent must stay well-behaved; a hung agent that keeps beating looks alive

PULL — orchestrator polls agent

- Orchestrator actively checks each agent
- + Simpler, more trustworthy failure detection
- – Scales worse as agent count grows

4.1 Interval & TTL Design

- **Too frequent** → noisy streams, wasted Redis throughput
- **Too infrequent** → slow failure detection, stale work assignment
- **Rule of thumb: $TTL \approx 2-3 \times$ the heartbeat interval**, tuned to the cost of reassigning work
 - Cheap reassignment → shorter TTL, faster detection
 - Expensive reassignment → longer TTL, tolerate transient misses

4.2 OpenClaw-Specific Mechanics

- **Transport:** dedicated Redis Stream per agent, or a **TTL'd key per agent** that expires if not refreshed
- **Detection loop (orchestrator side):** identify a dead / unreachable agent → trigger reassignment of its work
- **Flapping agents need debounce logic:**
 - Require **N consecutive misses** before declaring an agent “down”
 - Single-miss detection causes **thrashing** — agents oscillate between up/down and work gets needlessly reassigned

4.3 Edge Case: NanoClaw Agents

- **Intermittent connectivity** on Pi-class hardware breaks naive heartbeat assumptions
- **Exponential backoff** on reconnect attempts — don't hammer the orchestrator after a dropout
- **Last-known-state caching** — the orchestrator doesn't prematurely kill / reassign work during a transient blip
- **The core trade-off:**
 - **Aggressive detection** → fast reassignment, but more false positives
 - **Lenient detection** → fewer false positives, but slower recovery

4.4 Hands-On Lab: Kill and Detect

- 1. Write an agent process that pushes a heartbeat entry every **N** seconds
- 2. Write an orchestrator loop that flags an agent “down” after **M** consecutive missed beats
- 3. Kill the agent process mid-session and **measure detection time**
- 4. Adjust N and M, re-test — discuss the resulting **trade-off curve**

Discussion: what N / M values would you choose for a cloud-hosted agent vs a NanoClaw edge agent — and why?



PART 5 · 60 MIN

Skill Writing — Deep Dive

Skills are the agent's capabilities — and the description is the interface, not documentation.



5.1 Anatomy of a Skill

- Every skill has **two halves**:
 - **Metadata** — name, description, trigger conditions
 - **Execution logic** — the code that actually runs

“The description is not documentation — it is the interface the agent's tool-selection step reads.”

If the description is wrong, the skill may never fire — no matter how good the code is.

5.2 Writing for Discoverability

BAD

“Handles data stuff”

Vague descriptions cause **missed or misfired** skill invocations.

GOOD

“Fetches current weather for a given city and date; use only when the user asks about weather conditions, not climate history.”

Specific scope + explicit **when-to-use / when-not-to-use** boundaries.

- **Exercise:** critique 3–4 deliberately weak skill descriptions and rewrite them.

A good description names the action, the inputs, and the boundaries of when it applies.

5.3 Security Angle (OWASP Agentic Applications)

- **Sandboxing execution** — isolate skill code so a malicious or buggy skill can't escalate privileges
- **Input validation** — validate **agent-generated parameters** before execution (the model is an untrusted caller)
- **Output validation** — sanitize skill output **before it re-enters the agent's context / loop**

Reference: OWASP Agentic Applications Cheat Sheet

5.4 Hands-On Lab: Make It Fire (and Only When It Should)

- 1. Write a simple skill (e.g. a unit converter) with a clear description
- 2. Register it and confirm **correct invocation** on a matching prompt
- 3. “Vague-ify” the description and re-test — observe **misfire / no-invoke**
- 4. Restore the description, add basic **input validation**, try an adversarial input — confirm rejection

PART 6 · 60 MIN

Channel Integration — Messaging & Voice

Telegram, WhatsApp and voice: getting real users connected to your agents — and handling each channel's failure modes.

6.1 Messaging Channels: Telegram & WhatsApp

- **Webhook vs long-polling ingestion** — how an inbound message becomes an event on the OpenClaw Redis Stream
- **Session mapping** — map chat / session ID → agent instance so multi-turn conversation state stays coherent
- **Channel-specific quirks** — message length limits, rich media (images, voice notes sent as files), delivery / read receipts
- **Rate limits** — both platforms throttle aggressively; send replies with retry / backoff
- **Demo:** inbound Telegram message → OpenClaw agent → skill invocation → reply back to the same chat

6.2 Voice Channels

- **Two integration patterns:**
 - **Turn-based:** record → transcribe → agent → synthesize → play
 - **Streaming:** continuous ASR / TTS with **barge-in** support
- **ASR / TTS sit outside Hermes** — transcription happens before the model call, synthesis after; the agent still reasons over text
- **Latency budget** — voice has a much tighter round-trip tolerance than messaging; this drives the **cloud vs NanoClaw-local inference** choice
- **Interruptions / barge-in and silence detection** — a UX problem, not just a technical one
- **Demo / walkthrough:** a short voice round-trip through the same OpenClaw agent from the messaging demo

6.3 Channel Abstraction in OpenClaw

- Channels are a **thin adapter layer**, not baked into agent logic — the agent shouldn't know if it's talking to Telegram, WhatsApp, or a mic
- **Normalize inbound events** (text, voice-transcribed-to-text, media) into **one internal message format** before they hit the Redis Stream
- **Intersection with heartbeats (Part 4)**: the channel adapter itself needs liveness tracking — a dead webhook listener is as bad as a dead agent

6.4 Hands-On Lab: Wire a Channel

- 1. Wire one messaging channel (Telegram or WhatsApp sandbox / test account) to your Part 1 OpenClaw agent
- 2. Send a message that triggers the Part 5 skill; confirm the **reply round-trips** correctly
- 3. Discuss: what would need to change to add a **second channel** without touching agent logic?



PART 7 · 60 MIN

A2A Setup — a One-of-its-Kind Protocol

Designing a custom agent-to-agent protocol that is natively stream-based, heartbeat-aware and channel-agnostic.



7.1 Why Not Just Use an Off-the-Shelf A2A Standard?

- **The emerging A2A landscape:** agent-card discovery, task-based request / response protocols
- They assume infrastructure OpenClaw doesn't need: **HTTP-centric, synchronous request / response**
- **OpenClaw's differentiator: Redis Streams as the transport**
 - Durable, replayable, multi-consumer messaging **for free** — most A2A protocols bolt this on top of plain HTTP
- **Design goal:** an A2A layer that is natively **stream-based, heartbeat-aware** (Part 4) and **channel-agnostic** (Part 6) — not a generic point-to-point RPC

7.2 Anatomy of the Custom Protocol

- **Message envelope:** sender agent ID · recipient agent ID (or broadcast topic) · task ID · payload · **correlation ID** for multi-turn task tracking
- **Stream topology:** one stream per agent inbox **vs** shared topic streams with consumer groups
 - **Fan-out** (broadcast to many agents) vs **fan-in** (many agents reporting to one orchestrator) trade-offs
- **Task lifecycle states:** proposed → accepted → in-progress → completed / failed
 - Mirrored as stream entries — any consumer can **reconstruct task history from the stream itself** (replayability as a debugging feature)
- **Key difference:** typical A2A treats discovery & negotiation as separate from transport; here they're **unified — the stream itself is the source of truth**

7.3 Integrating Heartbeats and Skills into A2A

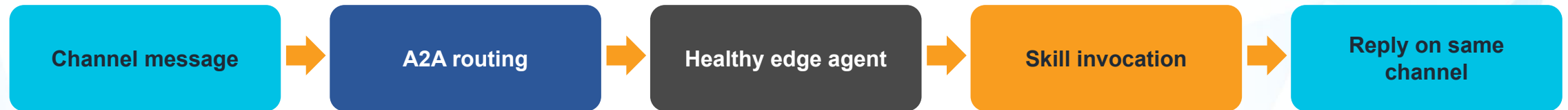
- **Reuse the Part 4 heartbeat stream as a capability-advertisement channel**
 - The heartbeat payload includes which **skills (Part 5)** the agent currently has registered → the orchestrator's routing decision uses **one lookup instead of two**
- **Routing logic:**
 - Incoming task → match required skill → find agents advertising that skill → filter to agents with a **live heartbeat** → dispatch
- **Failure handling unique to this design:** tasks live on a replayable stream, so a newly-spun-up agent (or a recovering NanoClaw device) **catches up on missed assignments by re-reading from its last acknowledged offset**

7.4 Hands-On Lab: Kill, Restart, Resume

- 1. Define the **message envelope schema** for the custom A2A protocol
- 2. Implement a two-agent exchange: Agent A proposes a task; Agent B accepts and reports progress via stream entries
- 3. Kill and restart Agent B **mid-task** — confirm it resumes from the stream rather than losing task state
- 4. Discuss: what would break if you swapped this for a standard **synchronous** A2A protocol? What would you lose — and what would you gain?

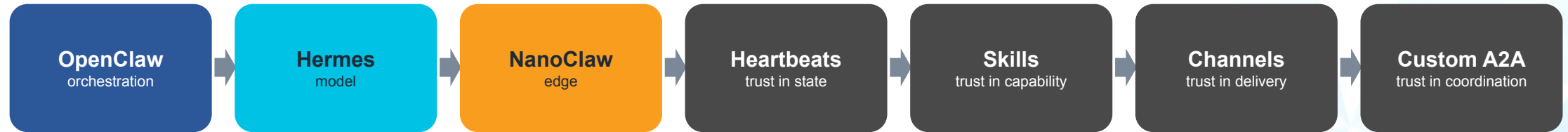
Combined Lab: End-to-End Channel-to-Skill Flow

Goal: a message (or voice input) arrives over a real channel → routed via the custom A2A protocol to a NanoClaw edge agent running Hermes → invokes a registered skill — while the OpenClaw orchestrator only dispatches to agents with **healthy heartbeats** advertising the **right capability**.



- 1. Confirm your Part 6 channel-wired agent is the same NanoClaw + Hermes agent from Part 3 — with the Part 5 skill, Part 4 heartbeat producer and Part 7 A2A envelope handling in place
- 2. Send a channel message that requires the skill to be invoked
- 3. Orchestrator uses Part 7 routing (skill match + live heartbeat) to select and dispatch to the edge agent
- 4. Simulate the edge agent going down mid-task — confirm the orchestrator reroutes / queues, and a recovering agent resumes from the stream
- 5. Discuss: how does replayability change “what happens to an in-flight task” — and how does that surface to the user over **messaging vs voice** differently?

Wrap-Up: The Seven-Layer Story



- Open Q&A
- Stretch topics for advanced learners:
 - A combined **health + capability registry** so orchestrator routing uses one signal instead of two (foreshadowed in 7.3)
 - **Streaming voice with barge-in** as a follow-up deep dive
 - **Benchmarking the custom A2A protocol** against a standard synchronous A2A implementation

Materials Checklist

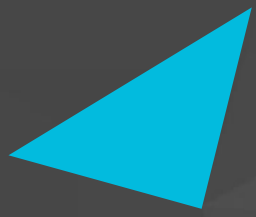
- **Redis instance** (local or shared) — orchestration, heartbeat and A2A stream labs
- **NanoClaw device** or Pi-class emulation environment
- **Hermes model weights** — quantized build suitable for the edge demo
- **Starter OpenClaw agent scaffold** with Redis Streams + MCP pre-wired
- **3–4 deliberately weak skill descriptions** for the critique exercise
- **Telegram and/or WhatsApp sandbox / test account** with webhook access
- **Basic ASR / TTS setup** (cloud API or local model) for the voice demo
- Reference: **OWASP Agentic Applications Cheat Sheet**
- Reference: **overview of at least one standard A2A protocol spec** (for the Part 7.1 comparison)



SECTION · INSTRUCTOR REFERENCE / HANDOUT

Technical Appendix

Configuring A2A + Hermes with LangGraph — implementation-level detail for learners who want to build past the guided demo.



A.1 Why LangGraph Fits Here

- Each OpenClaw agent's internal reasoning loop — **receive task** → **call Hermes** → **optionally invoke a skill / tool** → **respond** — maps naturally onto a LangGraph **StateGraph**
 - Nodes: “call model”, “invoke skill”, “await tool result”
 - Edges: conditional routing based on whether Hermes emitted a tool call
- LangGraph's built-in **checkpoint** lines up with the Part 7.2 task lifecycle (proposed → accepted → in-progress → completed / failed)
 - A checkpoint after each node execution gives the same **replayability** benefit as reading task state back from the Redis Stream

A.2 Wiring Hermes into a LangGraph Node

- Hermes isn't a first-party LangChain / LangGraph integration → serve it behind an **OpenAI-compatible endpoint** (vLLM, or Ollama / TGI exposing `/v1/chat/completions`)
- Bind it via **ChatOpenAI-style config** pointed at that local endpoint
- Confirm Hermes' tool-calling output matches what LangGraph expects (**OpenAI-style `tool\_calls`**) — this is the thing **most likely to need a translation shim**
- Bind Part 5 skills to the graph as **LangGraph tools** — the tool's `description` field is exactly the skill description from 5.2; **the same discoverability rules apply**

A.3 Connecting the Graph to the Redis Streams Transport

- The graph handles a **single agent's internal turn**; the Part 7.2 envelope (sender, recipient, task ID, payload, correlation ID) becomes the **input state** that triggers a graph invocation, and the **output state** serialized back onto the outbound stream
- **Pattern:** a thin consumer loop reads new entries from the agent's inbox stream → maps each to a LangGraph input state → invokes the graph → writes the result back as a new stream entry (status update or final response)
- The graph **doesn't need to know about Redis at all** — the two concerns stay cleanly separated
- Use the checkpointer with a **Redis-backed implementation** so an in-progress task's graph state survives an agent restart — the 7.3 “recovering agent resumes from stream offset” behaviour now also resumes **mid-graph-execution**, not just mid-task

A.4 Heartbeats & Capability Advertisement from Inside the Graph

- The graph **doesn't own the heartbeat loop** — keep heartbeat emission (Part 4) as a **separate lightweight process** alongside the graph consumer, not a graph node
 - It must keep beating even while the graph is mid-execution on a long task
- The heartbeat process can **introspect the graph's currently-bound tools** to build the capability-advertisement payload from 7.3
 - Skill registration stays a **single source of truth** (the graph's tool list) rather than a separately maintained config

A.5 Suggested Demo / Lab Extension

- 1. Stand up Hermes behind an OpenAI-compatible local endpoint
- 2. Build a two-node LangGraph (model call → conditional tool invocation) using one Part 5 skill as a bound tool
- 3. Wrap it with the thin Redis Streams consumer / producer loop from A.3
- 4. Re-run the Part 7.4 kill-and-resume test — confirm the **LangGraph checkpoint** (not just the task ID) is what allows correct resumption
- 5. Compare latency / complexity against the **framework-free** agent loop built in the main Part 7 lab

Thank You — Questions?



Take-home extensions: the four independent labs (heartbeat producer/consumer, skill critique-and-rewrite, channel wiring, A2A message envelope) + a follow-up review session.